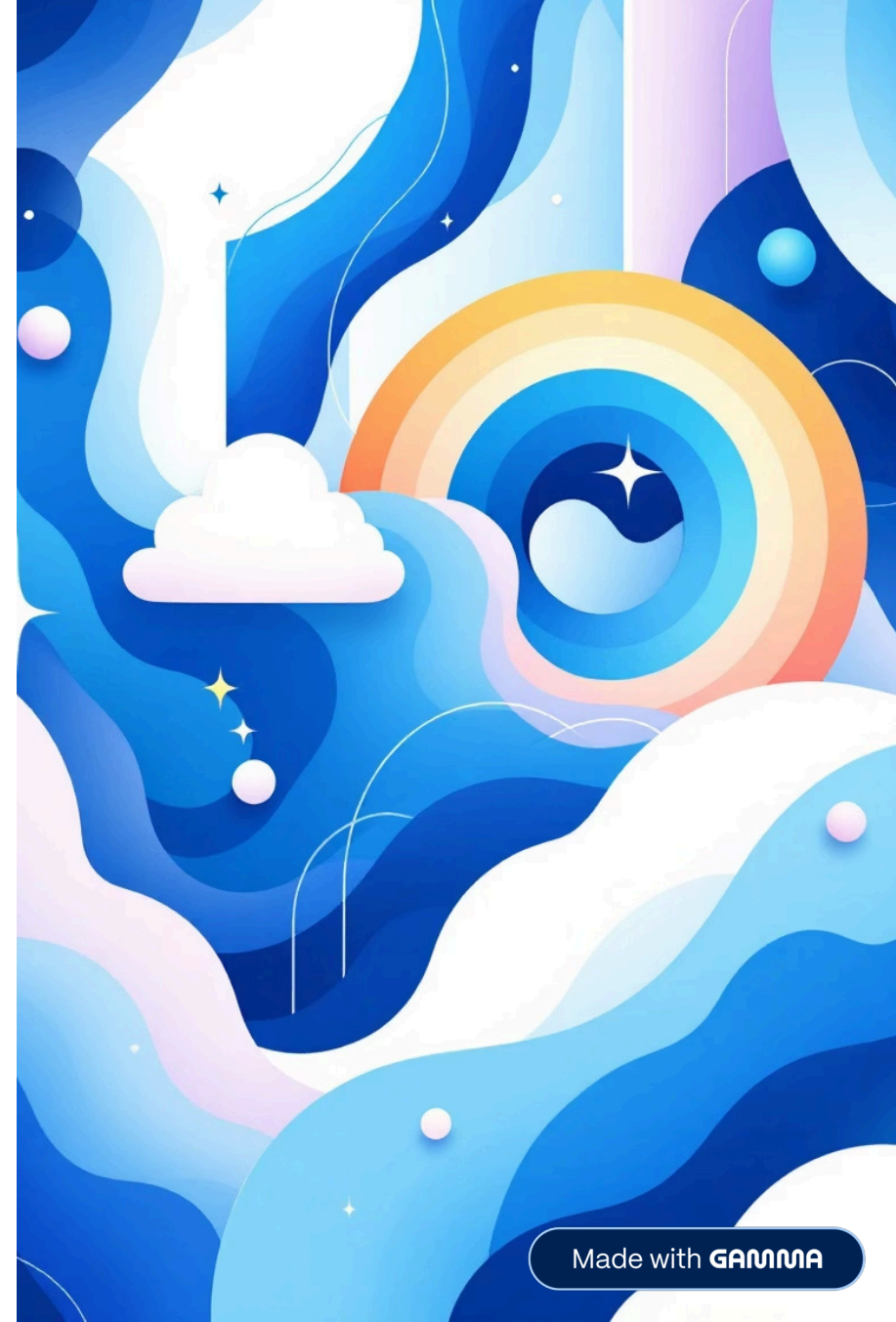


Операционные системы: назначение, виды и применение в разработке

Комплексное руководство для студентов и начинающих
разработчиков



Что такое операционная система?

Операционная система (ОС) — это комплекс взаимосвязанных программ, выступающий посредником между аппаратным обеспечением компьютера и пользователем.

Без ОС компьютер — просто набор электронных компонентов, неспособный выполнить ни одну полезную задачу.



- Для разработчика понимание ОС критически важно — это среда, в которой создаются и работают программы. Выбор ОС влияет на инструменты, производительность и совместимость.

Ключевые функции операционных систем



Управление процессами

Запуск, завершение и планирование процессов. Изоляция процессов друг от друга для обеспечения стабильности системы.



Управление памятью

Распределение оперативной памяти между процессами, защита и виртуализация памяти для эффективной работы.



Файловая система

Организация данных в логическую структуру. Обеспечение сохранности и быстрого доступа к файлам.



Управление устройствами

Унифицированный интерфейс для работы с оборудованием через систему драйверов.



Пользовательский интерфейс

GUI (графический) и CLI (командная строка) для взаимодействия с системой.



Безопасность

Управление учётными записями, правами доступа и защита от вредоносных программ.



Windows: универсальность и массовость

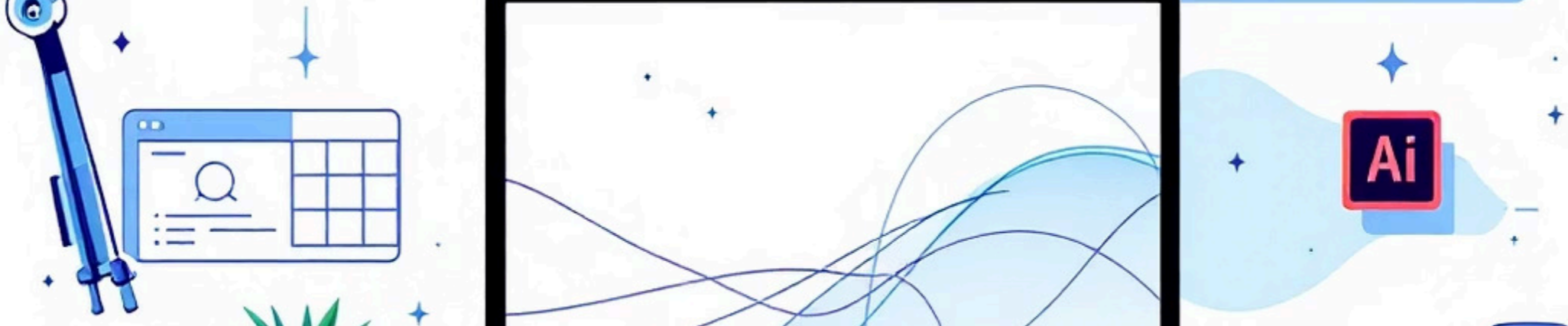
Философия

- Простота использования
- Обратная совместимость
- Доминирование на рынке (75-80%)
- Закрытая проприетарная система

Применение в разработке

- Фронтенд-разработка для массовой аудитории
- Геймдев с DirectX и игровыми движками
- Корпоративные решения (.NET, C#, MS SQL)
- Приложения для Microsoft Store

❑ **WSL (Windows Subsystem for Linux)** значительно улучшила возможности для бекенд-разработки, позволяя запускать Linux-программы прямо в Windows.



macOS: элегантность и интеграция

Философия Apple

Тесная интеграция аппаратного и программного обеспечения. Система «просто работает» благодаря оптимизации на глубоком уровне.

iOS-разработка

Обязательное требование: разработка приложений для iPhone и iPad возможна ТОЛЬКО на macOS с использованием Xcode.

Веб-разработка

UNIX-основа обеспечивает доступ к тем же инструментам, что и Linux, плюс качественные приложения для дизайна (Sketch, Figma).

Fullstack-разработка

Идеальная платформа для работы с бекендом (Node.js, Python), фронтендом (React), базами данных и дизайном на одном устройстве.

Linux: свобода, гибкость и контроль



Linux — не одна ОС, а **семейство дистрибутивов** на основе ядра Linux. Открытое программное обеспечение с лицензией GPL.

Популярные дистрибутивы: Ubuntu, Debian, Fedora, CentOS, Arch Linux.

01

Бекенд-разработка и DevOps

90% веб-серверов работают на Linux. Основная среда для Docker, Kubernetes, Ansible.

03

Data Science и AI

TensorFlow, PyTorch, научные библиотеки Python работают лучше всего в Linux.

02

Встроенные системы

От роутеров до автомобильных систем — везде Linux.

04

Android-разработка

Низкоуровневые задачи и кастомные сборки Android проще делать в Linux.

Выбор ОС для разных специализаций

Фронтенд-разработчики

Windows или macOS.

Windows для тестирования на платформе большинства пользователей, macOS для качественного железа и инструментов дизайна.

Бекенд-разработчики

Linux или macOS.

Разработка в среде, максимально похожей на серверную. Windows с WSL также подходит.

Мобильные разработчики

iOS: только macOS. **Android:** любая ОС. **Кросс-платформа:**

предпочтительно macOS для работы с обеими платформами.

Data Scientists

Linux для серверов, macOS для рабочих станций. Windows с WSL и специализированными инструментами также используется.

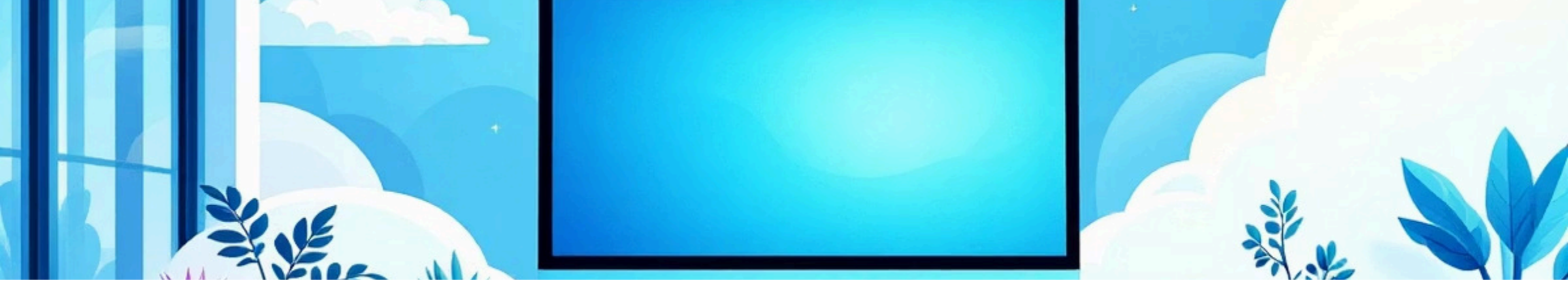
DevOps-инженеры

Почти 100% Linux.

Критически важно умение работать с командной строкой и писать bash-скрипты.

Разработчики игр

Windows для ПК-игр (Unity, Unreal Engine). Для консолей — Windows со специальными SDK. Для мобильных игр — по платформе.



Современные тенденции в мире разработки



Кросс-платформенность

VS Code, Git, Docker, большинство языков программирования работают одинаково на всех ОС



Виртуализация

WSL2, Docker, виртуальные машины стирают границы между операционными системами



Облачная разработка

GitHub Codespaces, Gitpod — разработка в браузере, ОС становится менее важной

Технологии виртуализации и облачные инструменты дают разработчикам беспрецедентную свободу выбора. Локальная ОС теперь менее критична для многих задач.

Практические рекомендации для начинающих

1 Начните с того, что есть

Не нужно сразу покупать Mac или переустанавливать систему. Начинайте учиться на текущем компьютере.

2 Освойте командную строку

PowerShell/WSL на Windows, Terminal на macOS/Linux — критически важный навык для любого программиста.

3 Экспериментируйте

Установите Linux на виртуальную машину (VirtualBox). Попробуйте разные среды, чтобы понять их философию.

4 Выбирайте ОС под задачу

iOS-разработка → Mac. Корпоративный .NET → Windows. DevOps/бекенд → Linux.

5 Инструменты важнее ОС

Квалификация определяется знаниями и умением решать задачи, а не операционной системой.

Заключение: свобода выбора и мастерство



Windows доминирует на десктопах и в корпоративной среде

macOS — стандарт для мобильной и творческой разработки

Linux правит миром серверов, облаков и встроенных систем

Операционная система — это всего лишь инструмент. Настоящее мастерство заключается не в выборе инструмента, а в умении им пользоваться для создания решений, которые делают мир лучше.

Время привязки к одной ОС уходит в прошлое. Кросс-платформенные инструменты, виртуализация и облачные технологии дают беспрецедентную свободу. Используйте эту свободу разумно, продолжайте учиться и не бойтесь экспериментировать!